

AI Audit Report — HW04 Automation Testing

Sinh viên: 23127195 **Ngày:** 2026-08-15 → 2026-08-16 **Múi giờ:** UTC+7 (giờ ghi bên dưới là giờ local; các mốc ISO trong HTML report là UTC)

Declaration: *I use AI tools for the following tasks.*

Công cụ AI đã sử dụng

Công cụ	Phiên bản / Model	Mục đích
Claude Code (CLI trong VS Code)	Claude Opus 5 (<code>claude-opus-5</code>)	Đọc đề, khảo sát SUT, sinh khung Playwright, sinh spec, phân tích lỗi, soạn tài liệu

Không sử dụng ChatGPT, Gemini, Copilot hay Cursor trong bài này.

Công cụ không phải AI

Playwright 1.62.1, Node.js 22.20.0, Git, SQLite (qua `sqlite3` của SUT).

Nhật ký tương tác

Cột "AI output" ghi tóm tắt kết quả và **tên artifact** sinh ra; toàn văn mã nguồn nằm trong repository, đây là bản ghi quá trình chứ không phải bản sao mã.

#1 — 11:32 | Đọc và bóc tách yêu cầu đề bài

Prompt:

"đọc file 2026.HW04.Automation .pdf cho tôi để nắm hết những yêu cầu, nắm kỹ cấu trúc nộp bài đi"

AI output: Tóm tắt 17 mục của đề: ngưỡng ≥12 test case/feature, dữ liệu phải tách file `.csv` / `.json` , ≥3 assertion pattern, ≥3 browser (≥9 run), HTML report phải hiện `Run by: {StudentID}` + ISO timestamp, video ≥5 phút tiếng Việt, ≥8 commit chạm file `.spec.js` , và

danh sách 9 mục bắt buộc trong file `.zip`.

Human review: Đối chiếu lại với PDF, xác nhận đúng. Ghi nhận hai điểm dễ mất điểm: báo cáo phải có cả `.md` lẫn `.pdf`, và `Run by:` phải nhúng vào report trước khi chạy chứ không thêm được sau.

#2 — 11:38 | Gộp SUT vào repository chính

Prompt:

"HW04-TESTING đang là github chính của tôi, còn eshop-sut là tui git clone 1 dự án về, làm sao để git add . hay gì đó để đẩy cả dự án eshop-sut lên github chính của tôi"

AI output: Chẩn đoán `eshop-sut` có `.git` riêng nên Git ghi nó thành gitlink (`mode 160000`) thay vì ghi file. Đề xuất: di chuyển `.git` của SUT ra ngoài repo (backup, không xoá), `git rm --cached -f eshop-sut`, tạo `.gitignore` giữ lại `playwright-report/`, rồi commit + push. Kết quả: commit `756fa65`, 67 file.

Human review: Chấp nhận. Yêu cầu backup thay vì xoá `.git` — đã làm đúng (`_eshop-sut_git_backup`). Kiểm tra lại `.gitignore` để chắc chắn không loại nhầm `playwright-report/`, vì đề bắt buộc HTML report phải nằm trong repo public.

#3 — 11:45 | Khai báo feature và giao việc FR-01

Prompt:

"Đây là các feature từ hw02 của tôi: FR-01 — ĐĂNG KÝ TÀI KHOẢN, FR-09 — MÃ GIẢM GIÁ, FR-14 — QUẢN LÝ DANH MỤC [...] bạn hãy làm cho tôi trước đầy đủ Feature 01 [...] nhớ là phải chạy dự án eshop-sut [...] và tạo thư mục 23127195 để viết báo cáo theo yêu cầu"

AI output: Lập kế hoạch 7 bước: cài & chạy SUT → khảo sát DOM → dựng config Playwright → viết data + spec → chạy 3 browser → viết báo cáo → commit.

Human review: Đồng ý thứ tự. Nhấn mạnh bước "khảo sát DOM trước khi viết spec" — đây chính là bước về sau ngăn được lỗi selector.

#4 — 11:47 | Cài đặt và khởi chạy SUT

Prompt: (nội bộ, do AI tự thực hiện theo yêu cầu #3)

Đọc `setup_guide.md`, cài dependencies backend + frontend-web, seed DB, khởi chạy 2 server.

AI output: `npm install` cho `eshop-sut/backend` (135 gói) và `eshop-sut/frontend-web` (225 gói); `node database.js` → "Database initialized and seeded (Phase 2)"; backend `:3000` OK; Vite `:5173` OK.

Human review: Phát hiện Vite bind vào **IPv6** `:::1`, nên `127.0.0.1:5173` không kết nối được — cần dùng `localhost`. Ghi nhận để tránh nhầm lẫn khi cấu hình `webServer`.

#5 — 11:50 | Khảo sát DOM thật của trang đăng ký

Prompt:

"Viết một script probe để kiểm tra: `getByLabel` có hoạt động không, cấu trúc input ra sao, và hành vi thực tế khi nhập `Password123!`, `Password 123`, email `abc`, email trùng."

AI output (trích nguyên văn kết quả chạy):

```
getByLabel("Họ Tên") -> count=0
getByLabel("Email") -> count=0
getByLabel("Mật khẩu") -> count=0
container("Họ Tên") -> count=1, type=text
[spec-valid pwd] error: Mật khẩu quá yếu! ... KÝ TỰ ĐẶC BIỆT.
[space pwd] url: http://localhost:5173/login
[duplicate email] url: http://localhost:5173/login
[bad email] url: http://localhost:5173/login
```

Human review: Đây là bước quyết định chất lượng cả bài. Kết quả bác bỏ giả định mặc định của AI (`getByLabel`) và đồng thời lộ ra 3 lỗi ứng viên của SUT. Quyết định: dùng selector neo theo `<div>` bọc label; đặt kỳ vọng test theo **đặc tả**, không theo hành vi đang có.

#6 — 11:52 | Dựng khung Playwright

Prompt:

"Tạo `playwright.config.js` với 3 project chromium/firefox/webkit, HTML report hiện Run by: 23127195 kèm ISO timestamp, và `webServer` tự khởi động SUT."

AI output: `playwright.config.js` với `reporter: [['html', { title: 'HW04 ... Run by: 23127195 | <ISO>' }]]`, `metadata`, `webServer` cho cả `:3000` và `:5173`.

Human review: AI ban đầu gắn thêm `module.exports.STUDENT_ID = ...` vào chính đối tượng config — sẽ nhét key lạ vào schema của Playwright. Đã tách sang `tests/utils/env.js`.

#7 — 11:55 | Sinh data file và spec

Prompt:

"Viết data file CSV cho ma trận mật khẩu và JSON cho các nhóm còn lại; spec phải ≥ 12 test case, ≥ 3 assertion pattern, không hardcode dữ liệu."

AI output: `tests/data/fr01-password-rules.csv` (10 dòng), `tests/data/fr01-register-cases.json`, `tests/utils/csv.js`, `tests/pages/RegisterPage.js`, `tests/fr01-register.spec.js` (27 test case, 6 assertion pattern P1–P6).

Human review: Bản nháp đầu có 2 dòng CSV trùng nội dung (TC-06 và TC-10 cùng `Password abc`) — đã đổi TC-10 thành `Password123` để bổ sung bằng chứng cho BUG-01.

#8 — 11:59 | Chạy thử lần 1 trên Chromium → phát hiện PASS GIẢ

Prompt:

"Chạy thử trên Chromium trước để bắt lỗi script sớm."

AI output: 23 passed, 4 failed. Bốn test fail đều gắn `@bug`.

Human review — phát hiện quan trọng nhất của bài: TC-18 → TC-21 (email sai định dạng) **PASS**, mâu thuẫn trực tiếp với kết quả probe ở #5 (email `abc` được chấp nhận). Truy nguyên: `assertion expect(page).toHaveURL(/\/register$/)` khớp **tức thì** vì SPA chưa resolve xong `POST /api/register`. Bộ test đang **che giấu** BUG-02. Yêu cầu AI thay bằng assertion hỏi thẳng backend số dòng `users`.

#9 — 12:00 | Sửa assertion → lỗi thật lộ ra

Prompt:

"Thay assertion racy bằng kiểm chứng tất định qua `GET /api/admin/users`; đảo thứ tự assertion ở nhóm 1 để chờ banner lỗi trước."

AI output: Thêm `countAccountsByEmail()`; kết quả đổi thành `19 passed, 8 failed` — 4 test email chuyển từ xanh giả sang **đỏ thật**.

Human review: Xác nhận đây mới là kết quả phản ánh đúng chất lượng SUT.

#10 — 12:02 | Chạy 3 browser → lộ flake chỉ trên WebKit

AI output: Chromium 19/8, Firefox 19/8, **WebKit 18/9**. Test dôi ra là TC-24 (SQL injection payload) với lỗi `expect(res.ok()).toBeTruthy()` — đăng nhập thất bại.

Human review: Xác định đây là lỗi **của bộ test**, không phải của SUT: spec gọi `POST /api/login` ngay sau `register()` mà không chờ request của trang hoàn tất. TC-23 vô tình che được vì có thêm `page.evaluate()` tạo độ trễ; TC-24 không có nên đua thẳng, và WebKit chậm hơn nên lộ. Đã thêm `await expect(page).toHaveURL(/\/login$/)` trước khi gọi API → 3 engine đồng nhất 19/8.

#11 — 12:06 | Phát hiện assertion đỏ vì SAI LÝ DO

Prompt:

"Kiểm tra thông báo lỗi của từng test fail xem có đúng bản chất không."

AI output: 5/8 test fail với message *"admin login must succeed to inspect user rows"* — tức chúng đỏ vì helper hỏng, **không phải** vì BUG-02/BUG-03.

Truy nguyên: `setup_guide.md` ghi mật khẩu admin là `admin123`, trong khi `database.js:92` seed `Admin123!`. Do FR-02 cộng `login_attempts + 2` với ngưỡng khoá 3, chỉ 2 lần thử sai đã khoá tài khoản admin 180 giây, và trong lúc khoá thì mật khẩu đúng cũng bị từ chối.

Human review: Đây là bài học đắt giá — **một test đỏ chưa chắc đã đỏ vì lý do mình nghĩ**. Nếu không đọc message mà chỉ nhìn con số 19/8 (trùng khớp với lần chạy trước) thì đã kết luận sai hoàn toàn về BUG-02 và BUG-03. Sửa mật khẩu theo **seed script (source of truth)** thay vì theo tài liệu, thêm cache token để giảm số lần đăng nhập, reseed DB, chạy lại và **đọc lại từng message** để xác nhận.

#12 — 12:08 | Chạy lại sau khi sửa → xác nhận nguyên nhân đúng

AI output: Chromium/Firefox/WebKit đều `19 passed, 8 failed`. Message của từng test fail giờ khớp đúng defect:

```
[TC-18] email "abc" must not create a user row – Expected: 0
[TC-22] a duplicate e-mail must not create a second account – Expected: 1
[TC-26] expect(received).not.toBe("Password 123")
[TC-02] toHaveURL /\login$/ – Received "http://localhost:5173/register"
```

Human review: Đã đối chiếu từng dòng. Chấp nhận kết quả.

#13 — 12:11 | Sinh bằng chứng và tài liệu

Prompt:

"Viết script chụp ảnh bằng chứng cho từng bug, kiểm chứng report có `Run by:`, rồi soạn báo cáo chính, bug report, AI audit, AI critique, README."

AI output: `scripts/verify-report-banner.mjs` (3 ảnh report, đều xác nhận `runBy=true` + ISO), `scripts/capture-bug-evidence.mjs` (4 ảnh bug), `23127195/report/HW04_Main_Report.md`, `23127195/bug-report/BUG_REPORT.md`, tài liệu này, `AI_Critique.md`, `README.md`.

Human review: Kiểm tra ảnh chụp report thấy tiêu đề *"HW04 EShop Automation - FR-01 | Run by: 23127195 | 2026-08-15T05:08:39.944Z"* — đạt yêu cầu chống gian lận của đề. Kiểm chứng thêm BUG-05 bằng tay: token `role='user'` đọc được 52 dòng từ `/api/admin/users`.

#14 — 22:05 | Giao làm nốt FR-09 và FR-14

Prompt:

"làm tiếp cho 2 FR còn lại đi nói chung là làm đầy đủ hết, tất cả yêu cầu, có gì tạo github isse cho FR1 và các FR còn lại luôn"

AI output: Kế hoạch: cài + chạy `frontend-admin` (:5174) cho FR-14, đọc mã nguồn coupon và category, probe hành vi thật bằng API trước khi viết spec, rồi mới sinh test.

Human review: Giữ nguyên nguyên tắc đã rút ra ở FR-01 — **probe trước, viết spec sau**. Đây là bước đã cứu cả FR-01 khỏi kết luận sai.

#15 — 22:12 | Probe FR-09 và FR-14 bằng API

Prompt:

"Viết script probe kiểm tra: công thức tính coupon percent/fixed, biên min_order_amount, mã hết hạn, giới hạn lượt dùng; và với category: tên rỗng, phân quyền user thường, id không tồn tại, xoá danh mục đang có sản phẩm."

AI output (trích kết quả chạy):

```
SAVE10 percent, total 500000 200 {"discount_amount":-4500000,"final_amount":5000000}
SAVE10 at min boundary 300000 400 {"error":"Đơn hàng chưa đủ giá trị tối thiểu 300,000 đ
POST category with empty name 200 {"message":"Category created","id":4}
POST category as ROLE=user 200 {"message":"Category created","id":6}
PUT /categories/99999 200 {"message":"Category updated"}
DELETE category 1 still used 200 → products still referencing: 2
```

Human review: Phát hiện lỗi nghiêm trọng nhất của toàn bài (BUG-07): công thức `total × (1 - discount_value)` cho ra số giảm **âm**, khiến khách bị tính gấp 10 lần. Quyết định viết TC-18 dưới dạng **bất biến** (giảm giá không âm, thành tiền ≤ tổng gốc) thay vì chỉ so số cụ thể — bất biến bắt được cả những biến thể của lỗi mà case cụ thể có thể bỏ sót.

#16 — 22:30 | Sinh spec FR-09, chạy thử → 12 pass / 6 fail

AI output: `fr09-coupon.spec.js` (18 test case), `CheckoutPage.js`, `fr09-coupon-calculations.csv`, `fr09-coupon-cases.json`.

Human review: Kiểm tra message của 6 test đỏ — tất cả đúng bản chất. Nhưng phát hiện `readAmount()` báo `4500000` thay vì `-4500000`: hàm vừa bắt dấu trừ trong regex vừa nhân `-1`, gây khử dấu. Nếu không soi thì bug report đã ghi sai con số.

#17 — 22:38 | Sinh spec FR-14 → 15/17 test fail

AI output: `fr14-category.spec.js` (17 test case), `AdminCategoryPage.js`. Kết quả lần đầu: **15 fail**, phần lớn timeout ở `input[type="email"]`.

Human review: Đọc `frontend-admin/src/App.jsx` thì ô email **không có thuộc tính** `type`, chỉ có `placeholder="Email"`, và nút ghi `Login` chứ không phải "Đăng nhập". Đây là **lặp lại đúng sai lầm R-01** trên một màn hình khác — AI tiếp tục suy diễn theo form chuẩn thay vì theo mã nguồn. Sửa thành `getByPlaceholder` + `getByRole('button', { name: 'Login' })` → còn 10 fail.

#18 — 22:46 | Truy vết test fail không gắn @bug

Prompt:

"TC-07 không gắn @bug mà vẫn fail, kiểm tra tại sao."

AI output: TC-07 so khớp danh sách UI với backend, nhận được `["Laptop", "Phụ kiện", "", "", "Hacked By User", "Danh mục kiểm tra nút sửa", ...]`.

Human review: Hai nguyên nhân, đều là lỗi thiết kế test của tôi:

1. Các test kỳ vọng "phải bị từ chối" chỉ dọn dẹp ở nhánh thành công — đúng lúc chúng **bắt được bug** thì thao tác lại thành công và để lại dòng rác.
2. TC-07 phụ thuộc danh mục seed `"Điện thoại"`, thứ mà chính probe trước đó đã xóa → kết quả phụ thuộc thứ tự chạy.

Sửa: hàm `cleanupAdded()` dọn dẹp **bất kể pass hay fail**; TC-07 tự tạo danh mục riêng để kiểm chứng. → 8 pass / 9 fail, toàn bộ fail đều `@bug`.

#19 — 23:02 | Chạy 9 browser run → phát hiện hồi quy do chính bản vá

AI output: FR-01 19/8 ×3 ổn định. FR-09 tụt từ 12 pass xuống **6 pass**.

Human review: Truy nguyên: bản vá `readAmount()` ở #16 đã bỏ dấu phẩy khỏi character class. Playwright để locale mặc định **en-US** nên trang render `50,000` chứ không phải `50.000`; regex tách thành `50` và `000` rồi lấy phần cuối → ra `0`. **Chính bản sửa lỗi lại tạo ra lỗi mới**. Sửa lại cho nhận cả hai dấu phân cách → khôi phục 12/6 đồng nhất trên 3 engine.

#20 — 23:20 | Kiểm chứng toàn cục và sinh tài liệu

Prompt:

"Kiểm tra mọi test fail có đúng đều gắn @bug không, và 9 report có banner không."

AI output:

```
Total executions : 186    Passed : 117    Failed : 69
OK - mọi test fail đều được gắn @bug (không có fail ngoài ý muốn)
9/9 report: runBy=true, ISO timestamp hợp lệ
```

Human review: Đây là bước kiểm chứng bắt buộc trước khi công bố số liệu — sau bài học R-08, không chấp nhận kết luận từ con số tổng mà không quét từng test. Sau đó sinh ảnh bằng chứng cho 9 bug mới và cập nhật toàn bộ tài liệu.

#21 — 16/08, 12:30 | Demo Agent Skill trên FR-05

Prompt:

"Dùng skill eshop-automation để tự động hoá FR-05 — tìm kiếm sản phẩm [...] đặc biệt kiểm tra kỹ ô tìm kiếm có lỗ hổng XSS không."

AI output: Theo đúng 8 bước của skill: probe DOM + hành vi thật, rồi sinh `tests/fr05-search.spec.js` (21 test case), `SearchPage.js`, hai file dữ liệu và helper `searchProducts`. Chạy 3 engine ra 16/5 đồng nhất; 5 test đỏ là XSS phản chiếu và SQL injection thật ở ô tìm kiếm.

Human review: Bước probe cứu một bàn thua trông thấy. Payload thử đầu tiên là `<svg onload=...>` nhưng probe cho thấy nó **không chạy** khi chèn qua `innerHTML`, tức test sẽ xanh giả và bỏ sót lỗ hổng. Đổi sang `<img src=x onerror=...>` mới lộ ra XSS (ghi thành R-13). Ba bug này để riêng ở phần Agent Skill, không trộn vào 15 bug của ba feature chính. FR-05 chỉ là demo, không tính vào feature nộp.

#22 — 16/08, 13:10 | Hoàn thiện và đóng gói, phát hiện thêm một race

Prompt:

"hoàn thiện bài làm toàn diện [...] verify lại hết [...] chỉ cần nén file rồi nộp"

AI output: Bổ sung TC-19 cho FR-09 (gọi thẳng API thiếu `code` phải 400), gán provenance `Run by: 23127195` vào từng test, cập nhật lại toàn bộ số liệu, chèn hai link video, và dựng gói nộp.

Human review: Khi chạy lại ma trận, FR-14 tụt xuống 7/10 **chỉ trên Firefox**. Truy tới cùng: TC-02 đọc bảng UI trước khi React kịp render dòng mới (R-12). Đã sửa bằng cách chờ dòng mới hiện ra rồi mới đọc; Firefox chạy lại nhiều lần đều ổn định 8/9. Đúng tinh thần R-08: một con số lệch là đủ để phải truy, không bỏ qua.

Tổng kết mức độ can thiệp của con người

Hạng mục	Số lượng
Lỗi trong sản phẩm do AI sinh, được người phát hiện và sửa	13 (R-01 → R-13)
Trong đó lỗi khiến test xanh giả (nguy hiểm nhất)	3 (R-03, R-07, R-13)
Trong đó lỗi khiến test đỏ vì sai lý do	3 (R-08, R-10, R-11)
Trong đó lỗi chỉ lộ khi chạy đa trình duyệt	2 (R-05, R-12)
Trong đó bản vá lại tạo ra lỗi mới	1 (R-09)
Trong đó sai lầm lặp lại dù đã sửa một lần	1 (R-10 lặp lại R-01)
Quyết định thiết kế do người đưa ra, không phải AI	Đặt kỳ vọng theo đặc tả thay vì theo code; chấp nhận 23 test đỏ thay vì nói cho xanh; dùng assertion bất biến cho TC-18 của FR-09

Cam kết: Tôi đã đọc, kiểm chứng và chịu trách nhiệm hoàn toàn về mã nguồn và kết quả trong bài nộp này.